	Carátula para entrega de prácticas
Facultad de Ingeniería	Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): René Adrián Dávila Pérez.

Asignatura: Programación orientada a objetos.

Grupo: 07

No de Práctica(s): 7

Integrante(s): 323331229

426060019

323217156

312109239

323195386

321321976

No. de lista o brigada: 6

Semestre: 3ro

Fecha de entrega: 16/10/2026

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción **3**
1.1. Planteamiento del problema 3
1.2. Motivación 3
1.3. Objetivos 3

2. Marco Teórico **4**

3. Desarrollo **5**
3.1. Ejercicio 1. 5
3.2. Ejercicio 2. 5

4. Resultados **6**
4.1. Resultados del Ejercicio 1. 6
4.2. Resultados del Ejercicio 2. 7

5. Conclusiones **10**

1. Introducción

1.1. Planteamiento del problema

La practica nos solicita en el primer ejercicio preguntarle a nuestro LLM lo siguiente: ¿Por qué no es buena práctica usar encapsulamiento protected en herencia? Para posteriormente comparar las respuestas con otros LLM. Para nuestro segundo ejercicio nos solicitan diseñar e implementar una jerarquía de herencia lo más profunda posible para distintas figuras geométricas partiendo de una superclase Cuadrilatero con vértices privados basados en una clase auxiliar Punto, elaborar sus diagramas UML, desarrollar una aplicación principal que instancie objetos de cada clase para calcular y mostrar sus áreas específicas (omitiendo la del cuadrilátero base).

1.2. Motivación

Implementar los conceptos de herencia en programación orientado a objetos.

1.3. Objetivos

- **General:** Aprender a estructurar código mediante jerarquías de herencia profundas, aplicar la composición de objetos, abstraer el diseño con diagramas UML y comprender la importancia de proteger los datos mediante el encapsulamiento correcto.
- **Específicos:**
  - Preguntar a nuestro LLM: "¿Por qué no es buena práctica usar encapsulamiento protected en herencia?"
  - Crear una clase Punto para representar las coordenadas (x, y) de las figuras.
  - Crear la superclase Cuadrilatero. Esta debe contener como variables de instancia privadas los cuatro vértices de la figura, utilizando objetos de la clase Punto.
  - Crear las clases Trapezoide, Paralelogramo, Rectangulo y Cuadrado.
  - La jerarquía debe ser lo más profunda posible. Cuadrilatero → Trapezoide → Paralelogramo → Rectangulo → Cuadrado.
  - Especificar las variables de instancia y los métodos para cada una de estas clases.
  - Escribir el método main para crear instancias de objetos de cada una de las clases.
  - Calcular y mostrar en consola el área de cada objeto instanciado, omitiendo el cálculo de área para la clase base Cuadrilatero.

2. Marco Teórico

- **La clase Object:** Es la raíz de la jerarquía de clases.Toda clase tiene Object como superclase. Todos los objetos, incluidos los arreglos, implementan los métodos de esta clase. [2]
- **Herencia:** Permite crear nuevos objetos que asumen las propiedades de objetos existentes permitiendo agregar más funcionalidades. [1]
- **Súper clase:** Una clase que es usada como base para heredarse. [1]
- **Subclase:** Una clase que hereda de una súper clase. La clase derivada hereda todas las propiedades y métodos visibles de la clase base y, además, puede agregar propiedades y métodos propios. [1]
- **IS-A:** (es un) Se basa en la herencia y permite afirmar que un objeto es de un tipo (clase) en específico. [3]
- **HAS-A:** (tiene un) Se basa en el uso más que en la herencia, es decir, una clase X HAS-A Y si el código en la clase X tiene como atributo una referencia de la clase Y. [3]
- **Sobrescritura:** La sobrescritura se refiere a la habilidad de redefinir el comportamiento de un método específico en una subclase, generando así un comportamiento acorde a las necesidades de cada clase. [3]
- **toString:** Es definido dentro de la clase Object. Este método es utilizado para mostrar información de un objeto, para mostrar la información deseada es necesario sobrescribir el método. Debe tener el mismo nombre. Debe tener el mismo tipo y número de parámetros. El tipo de nivel de acceso debe ser igual o más accesible. El valor de retorno debe ser del mismo tipo o un subtipo. [3]
- **Super:** Dentro de una clase derivada se puede acceder a los elementos de la clase base a través de la palabra reservada super. [3]

3. Desarrollo

3.1. Ejercicio 1.

Esta actividad consistía en preguntar ¿Por qué no es buena práctica usar encapsulamiento protected en herencia? recibimos 3 respuestas por parte de Gemini, Claude y Chatgpt respectivamente donde destacan los siguientes puntos:

- Gemini: Señala que el uso de protected rompe directamente con el principio de encapsulación y genera un acoplamiento fuerte (tight coupling) entre la clase padre y sus subclases.
- Claude: Explica que al usar protected, los atributos internos se convierten accidentalmente en parte de la API pública. Además, resalta que en Java, el alcance de protected es más amplio de lo que parece, permitiendo el acceso a cualquier clase que comparta el mismo paquete, no solo a las clases hijas.
- ChatGPT Destaca que al tener acceso directo, las subclases pueden modificar atributos saltándose las reglas de validación (como asignar una edad negativa). Esto hace que el mantenimiento sea mucho más difícil, ya que cualquier cambio en los detalles internos de la clase padre obliga a modificar y revisar múltiples clases hijas.

La conclusión a la que llegan es que el uso de atributos protected crea una dependencia que vuelve al sistema inestable. Los tres coinciden en que la práctica correcta para mantener la integridad de los datos es declarar siempre las variables de instancia como private en la superclase, y proporcionar acceso a las subclases exclusivamente mediante (getters y setters), protegiendo el código base de modificaciones accidentales.

3.2. Ejercicio 2.

Para nuestro segundo ejercicio nos solicitan diseñar e implementar una jerarquía de herencia lo más profunda posible para las figuras geométricas (Trapezoide, Paralelogramo, Rectángulo y Cuadrado) partiendo de una superclase Cuadrilatero. cumpliendo con la instrucción de que la jerarquía sea lo más profunda posible.

- Se creó una clase `Punto` para manejar las coordenadas (x, y) en el plano cartesiano.
- Se definió la superclase `Cuadrilátero`, la cual al contener cuatro atributos privados `Punto`. Para acceder a estos vértices desde las clases hijas, se implementaron métodos *getters*.
- Se construyó una cadena de herencia lo más extensa posible utilizando la palabra reservada `extends`:`Cuadrilátero` → `Trapezoide` → `Paralelogramo` → `Rectángulo` → `Cuadrado`.
- En la superclase `Cuadrilátero` se declaró el método `calcularArea()` devolviendo cero. Este método fue sobrescrito usando la anotación `@Override` en cada una de las subclases para implementar la fórmula matemática correspondiente a cada figura, apoyándose en un método auxiliar para calcular distancias entre puntos.

- Se desarrolló la clase `Main`, en ella se instanciaron objetos de cada figura pasándoles coordenadas específicas y se implementó un menú interactivo mediante un ciclo `do-while` y una estructura `switch`. Este menú permite al usuario seleccionar una figura para visualizar sus vértices, mostrar su perímetro y calcular su área específica.

Como prueba en el main se instanció un `Trapezoide` con vértices en $(0,0), (8,0), (6,4), (2,4)$; un `Paralelogramo` en $(0,0), (6,0), (8,4), (2,4)$; un `Rectángulo` en $(0,0), (6,0), (6,4), (0,4)$; y un `Cuadrado` en $(0,0), (5,0), (5,5), (0,5)$.

4. Resultados

4.1. Resultados del Ejercicio 1.

Se muestran las respuestas obtenidas al consultar a los diferentes LLM sobre el por qué se considera una mala práctica utilizar el modificador de acceso `protected` en la herencia.

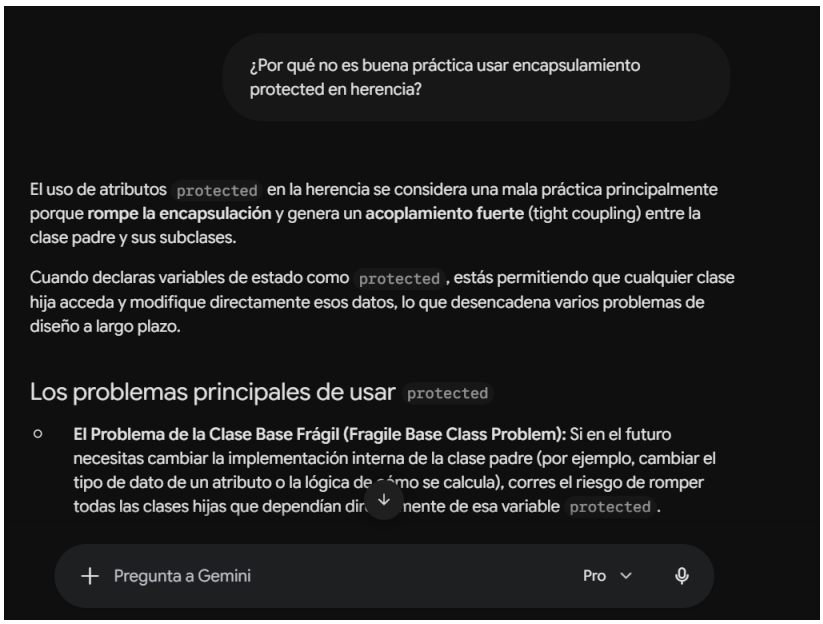


Figura 1: Respuesta proporcionada por Gemini.

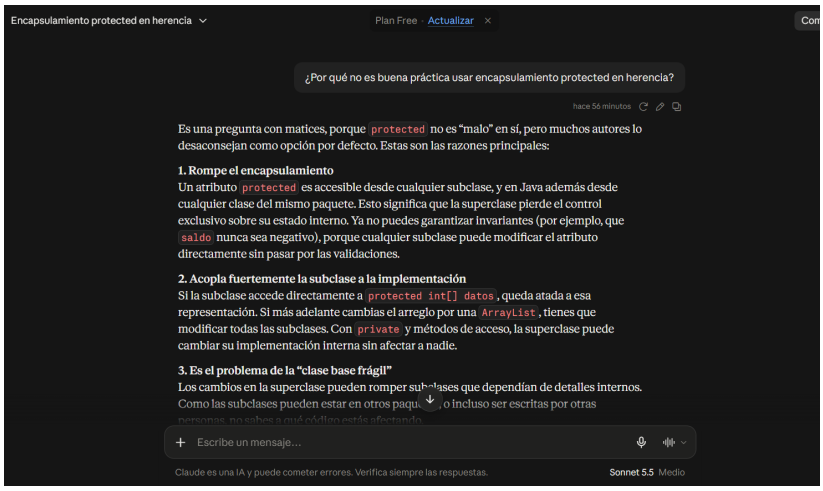


Figura 2: Respuesta proporcionada por Claude.

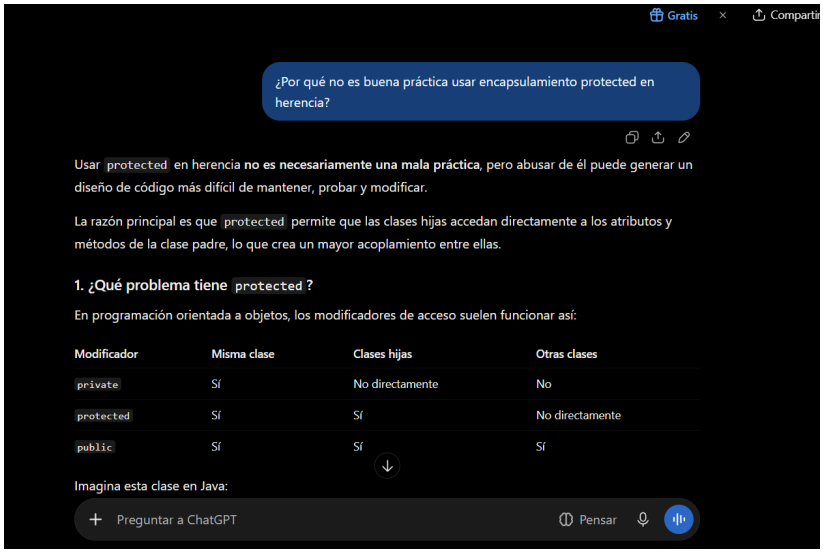


Figura 3: Respuesta proporcionada por ChatGPT.

4.2. Resultados del Ejercicio 2.

El diagrama UML que modela la jerarquía de herencia profunda y la composición de la superclase `Cuadrilátero` junto con sus respectivas subclases.

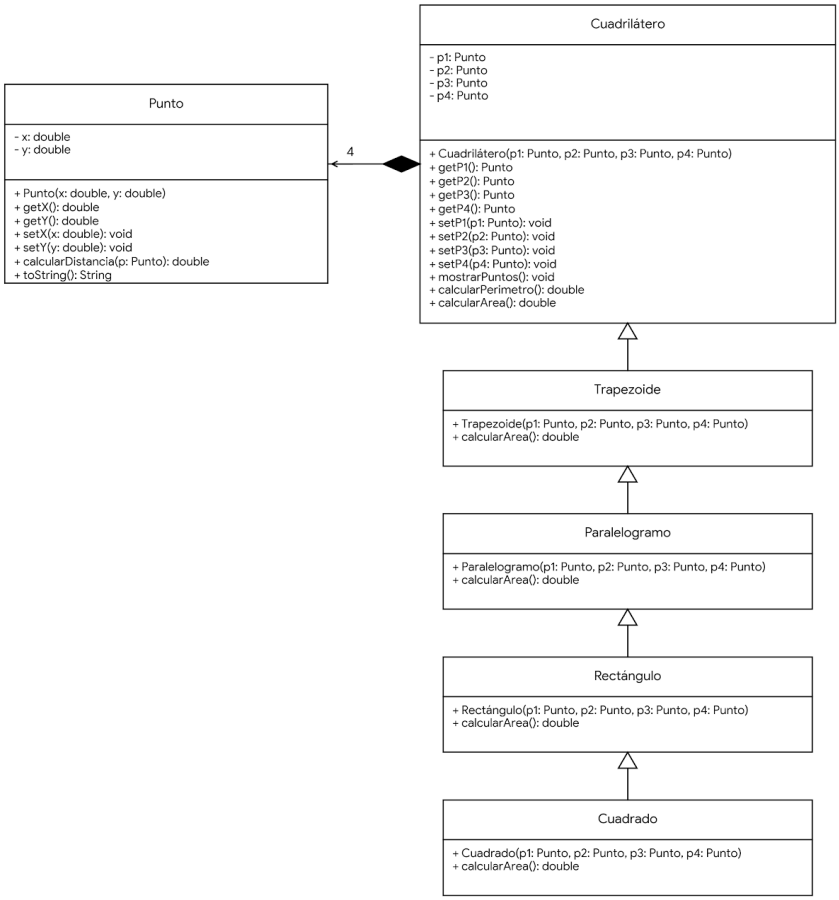


Figura 4: Diagrama UML representando la jerarquía y estructura de las clases.

Se realizó la prueba de ejecución del programa principal (`Main.java`) en la terminal. Se verificó el despliegue correcto del menú interactivo, se instanció un `Trapezoide` con vértices en $(0,0),(8,0),(6,4),(2,4)$; un `Paralelogramo` en $(0,0),(6,0),(8,4),(2,4)$; un `Rectángulo` en $(0,0),(6,0),(6,4),(0,4)$; y un `Cuadrado` en $(0,0),(5,0),(5,5),(0,5)$ y se comprobó que el cálculo de perímetros y áreas corresponde matemáticamente a los casos de prueba ingresados.

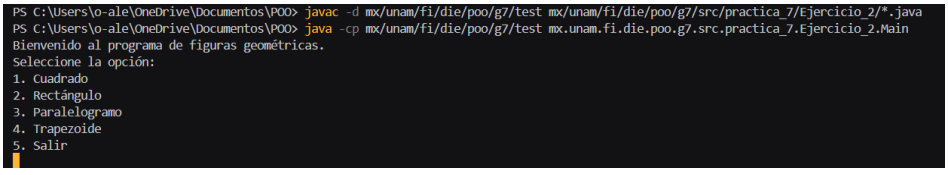


Figura 5: Ejecución del programa mostrando el menú principal y la selección de figuras.

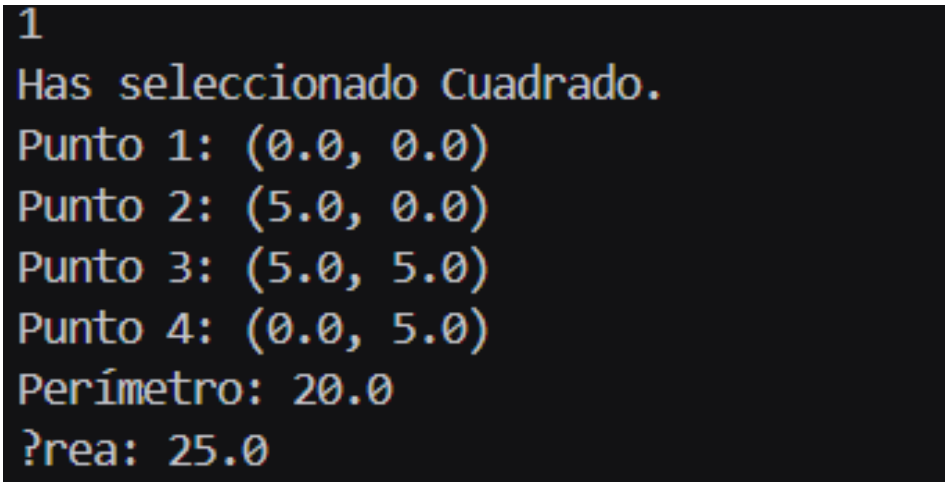


Figura 6: Resultados en consola mostrando los vértices, el perímetro y el área calculada del Cuadrado.

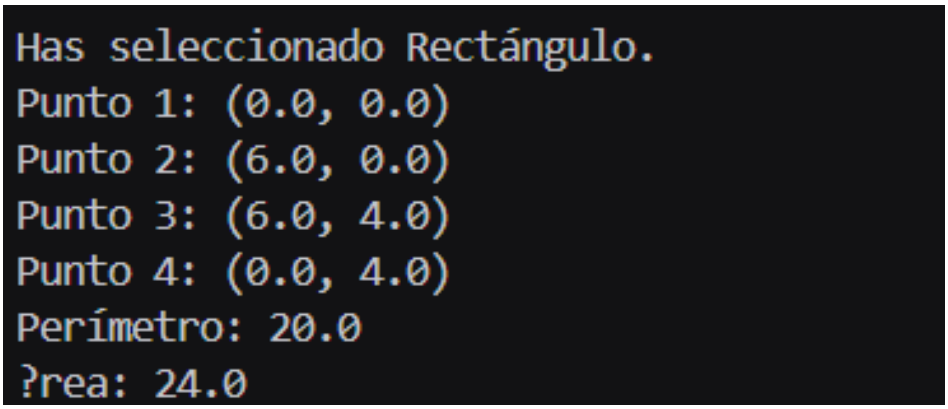


Figura 7: Resultados en consola mostrando los vértices, el perímetro y el área calculada del Rectángulo.

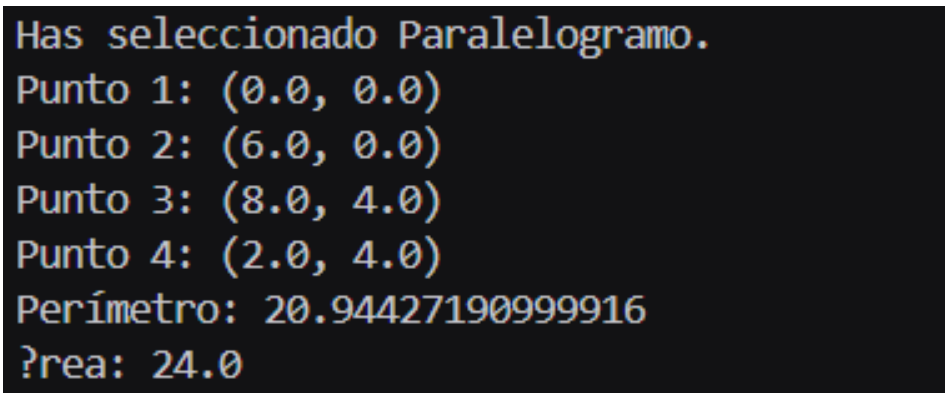


Figura 8: Resultados en consola mostrando los vértices, el perímetro y el área calculada del Paralelogramo.

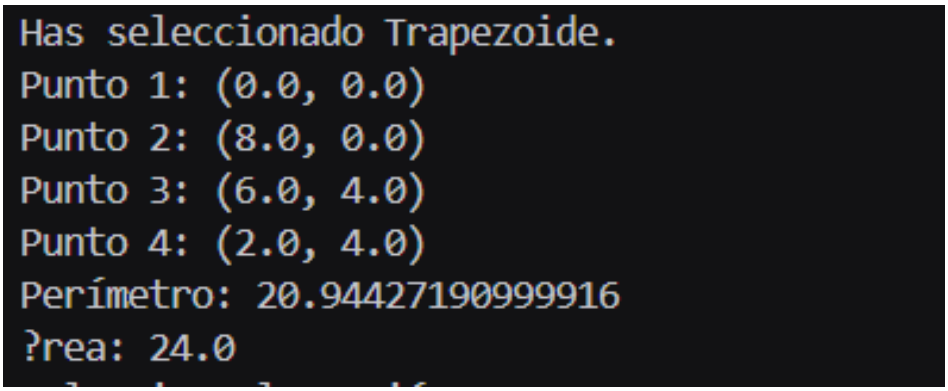


Figura 9: Resultados en consola mostrando los vértices, el perímetro y el área calculada del Trapezoide.

5. Conclusiones

A través de la comparación de respuestas de diferentes Modelos de Lenguaje, se comprendió que el uso de atributos `protected` debilita la encapsulación. Con la construcción de la jerarquía (`Cuadrilátero` → `Trapezoide` → `Paralelogramo` → `Rectángulo` → `Cuadrado`) es una clara demostración de cómo la herencia permite modelar relaciones lógicas del tipo IS-A. La sobreescritura del método `calcularArea()` permitió que el main interactuara con distintas figuras geométricas mientras que cada objeto respondió ejecutando su propia fórmula matemática según su naturaleza. El uso de la clase `Punto` como componente de la superclase comprobó que la herencia y la composición son complementarias.

Referencias

[1] Olguín, J. A. S. G. R. A. S. (s.f.-b). Herencia. Unidades de Apoyo Para el Aprendizaje - CUAED - UNAM. https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/3057/mod_resource/content/1/UAPA-Herencia/index.html

[2] Object (Java Platform SE 8). (2026, 21 julio). https://docs.oracle.com/javase/8/docs/api/java/lang/Object.html

[3] Programador Certificado Java 2. (2.a ed.). (1992). Alfaomega Grupo Editor.